COURSE-BASED PROJECT DOCUMENTATION | DOC-07

Lexical Analyzer (GNU Flex)

DFA Regular Expressions, Location Tracking, & Lexer States

1. Lexical Analysis Principles

Lexical analysis (scanning) is the first phase of a compiler. It converts a raw stream of characters into discrete, meaningful atomic units called tokens. GNU Flex compiles high-level regular expression specifications into a table-driven Deterministic Finite Automaton (DFA) executing in linear O(n) time.

2. Scanner Specification (lexer/java_lexer.l)

- Keywords: 50+ Java keywords (class, interface, record, sealed, permits, when, var, yield, etc.).
- Literals: Decimal, Hexadecimal (0x...), Binary (0b...), Octal, Floating-point, Char, String, Text Blocks.
- Unicode Identifiers: Supports ASCII, $, _, and multi-byte UTF-8 bytes ([\x80-\xFF]).
- Operators & Separators: All Java operators including >>>=, ::, ->, @, ..., etc.

3. Lexer Start Conditions (States)

Exclusive states (%x) handle multi-line comments and modern text blocks:

```
%x STATE_BLOCK_COMMENT
%x STATE_TEXT_BLOCK

/* Block Comment */
"/*"                      { BEGIN(STATE_BLOCK_COMMENT); }
<STATE_BLOCK_COMMENT>"*/"  { BEGIN(INITIAL); }
<STATE_BLOCK_COMMENT>\n    { current_column = 1; }
<STATE_BLOCK_COMMENT>.     { /* ignore */ }

/* Text Block */
"\"\"\""                  { BEGIN(STATE_TEXT_BLOCK); }
<STATE_TEXT_BLOCK>"\"\"\"" { BEGIN(INITIAL); return TEXT_BLOCK; }
```

4. Location Tracking via YY_USER_ACTION

```
#define YY_USER_ACTION \
    yylloc.first_line = yylloc.last_line = yylineno; \
    yylloc.first_column = current_column; \
    yylloc.last_column = current_column + yyleng - 1; \
    current_column += yyleng;
```

This hook guarantees that every token delivered to Bison carries exact line and column coordinates.